A Tutorial on Digit Recurrence Algorithms for Division and Square Root

Joonsang Yoon

January 30, 2026

Abstract

Digit recurrence algorithms, especially those in the SRT (Sweeney, Robertson, and Tocher) family, form a standard foundation for high-speed hardware division and square root. Their key idea is to compute the result one digit at a time while representing those digits in a redundant signed-digit set. Redundancy creates overlap between the valid selection regions of adjacent digits, which is the critical performance enabler: it allows the next digit to be selected using fast, truncated estimates of the internal state (the residual), rather than an exact residual value obtained by full-width carry-propagating addition. This tutorial develops the theory and practice of these methods from first principles. We derive the core recurrence relations for division and square root, establish the corresponding convergence/containment requirements, and show how those requirements translate into implementable digit-selection logic. To connect the mathematics to hardware design, we work through detailed Radix-2 and Radix-4 examples, including the determination of key precision parameters and the systematic derivation of final selection tables.

1 Introduction

Division and square root are fundamental arithmetic operations in modern microprocessors, with broad use in scientific computing, signal processing, and graphics. In contrast to addition and multiplication, division and square root are inherently iterative in hardware. Among the most prevalent high-performance approaches are *digit recurrence algorithms*.

A digit recurrence algorithm produces the result (a quotient or a square root) from most significant digit to least significant digit. Each iteration performs two tightly coupled actions:

1. select the next result digit, and
2. update an internal *residual* (partial remainder).

Because these actions repeat every cycle, overall performance is determined by the speed of this loop.

A direct implementation runs into a bottleneck during digit selection. To choose a correct digit, the algorithm must compare the current residual to certain multiples of the divisor (for division) or to quantities derived from the evolving partial root (for square root). These comparisons appear to require the *exact* residual value at full precision. If the residual were maintained in conventional binary form, computing it exactly each cycle would require full-width carry-propagating addition, slowing the critical path.

SRT algorithms avoid this bottleneck by selecting digits from a *redundant signed-digit set* (for example, $\{-1, 0, 1\}$ in Radix-2 rather than $\{0, 1\}$). Redundancy introduces overlap between the valid selection regions of neighboring digits. That overlap is the key performance enabler: even when the residual is known only approximately (via truncation), the selection logic can still choose a digit that keeps the recurrence within a safe region. In hardware, this makes it practical to keep the residual in carry-save form and base digit selection on a small number of leading bits, using only a short carry-propagate adder (CPA) and a small lookup table.

This tutorial develops the design methodology systematically:

- section 2 introduces redundant digit sets and the redundancy factor ρ, which quantifies the overlap that makes truncated selection possible.
- section 3 derives digit recurrence division: the residual recurrence, the containment requirement, and digit-selection intervals, followed by Radix-2 and Radix-4 selection logic derived using truncated operands.
- section 4 applies the same methodology to square root. We derive the corresponding recurrence and selection bounds, explain what changes relative to division, and work through Radix-2 and Radix-4 examples (including iteration-dependent startup constraints).
- section 5 summarizes the core principles and practical trade-offs.

2 Redundancy in Digit Sets

High-speed digit recurrence algorithms rely on representing the result digits in a redundant signed-digit set. For radix B, instead of using a single non-overlapping representation of digits, SRT-style methods select each digit from a symmetric set:

$$q_{j+1}, s_{j+1} \in \{-a, -a+1, \ldots, a-1, a\}, \tag{1}$$

where the integer a is the maximum digit magnitude.

Redundancy is quantified by the *redundancy factor*:

$$\rho = \frac{a}{B-1}. \tag{2}$$

The crucial requirement for fast digit selection is

$$\rho > \frac{1}{2}. \tag{3}$$

This inequality is essential because it guarantees that the digit-selection intervals for adjacent digits overlap. As derived later (e.g., in section 3), each digit k is valid only when the residual lies within a corresponding interval. When $\rho > \frac{1}{2}$, the interval for k overlaps the interval for $k+1$, so there exist residual values for which *either* digit leads to a valid next residual. That overlap is exactly what allows practical hardware to base digit selection on truncated residual estimates rather than on an exact full-precision residual. At the boundary $\rho = \frac{1}{2}$, overlap disappears (intervals merely touch), and selection becomes sensitive to exact comparisons.

In practice, ρ is typically chosen within

$$\frac{1}{2} < \rho \leq 1, \tag{4}$$

which implies the corresponding range for a:

$$\frac{B-1}{2} < a \leq B-1. \tag{5}$$

The lower bound ensures overlap. The upper bound is pragmatic: while larger a increases overlap, it also increases the complexity of forming $q_{j+1}D$ (or analogous terms), often with diminishing returns. The case $a = B-1$ (so $\rho = 1$) is *maximally redundant*, while choices near $a \approx \frac{B-1}{2}$ (so $\rho \to \frac{1}{2}$) are *minimally redundant*.

3 Division by Digit Recurrence

We begin with division. The fundamental relationship is

$$N = DQ_n + r, \tag{6}$$

where N is the dividend, D the divisor, Q_n the final quotient, and r the final (unscaled) remainder. For fractional arithmetic, it is common to normalize operands. We assume

$$N \in \left[\frac{1}{4}, \frac{1}{2}\right) \qquad \text{(Dividend)}$$

$$D \in \left[\frac{1}{2}, 1\right) \qquad \text{(Divisor)}$$

which yields a convenient quotient range:

$$Q_n \in \left(\frac{N_{\min}}{D_{\max}}, \frac{N_{\max}}{D_{\min}}\right) = \left(\frac{1/4}{1}, \frac{1/2}{1/2}\right) = \left(\frac{1}{4}, 1\right). \tag{7}$$

3.1 Quotient digits and approximation

The quotient is generated over n iterations, with $j = 0, 1, \ldots, n-1$. Let Q_j be the current quotient approximation after j digits $(q_1, \ldots, q_j)$ have been chosen. At each step, the next digit refines the approximation:

$$Q_{j+1} = Q_j + q_{j+1} B^{-(j+1)}. \tag{8}$$

Unfolding this recurrence gives

$$\text{At step } j, \quad Q_{j+1} = Q_0 + \sum_{i=0}^{j} q_{i+1} B^{-(i+1)}. \tag{9}$$

After n iterations,

$$\text{At step } j = n-1, \quad Q_n = Q_0 + \sum_{i=0}^{n-1} q_{i+1} B^{-(i+1)}. \tag{10}$$

3.2 Error bound and containment

At step j, define the approximation error

$$\varepsilon_{j+1} = \frac{N}{D} - Q_{j+1} = \sum_{i=j+1}^{\infty} q_{i+1} B^{-(i+1)}. \tag{11}$$

With $|q_{i+1}| \le a$,

$$\left| \sum_{i=j+1}^{\infty} q_{i+1} B^{-(i+1)} \right| \le \sum_{i=j+1}^{\infty} a B^{-(i+1)} = a B^{-(j+2)} \sum_{k=0}^{\infty} B^{-k} = a B^{-(j+2)} \frac{1}{1 - B^{-1}} = \frac{a}{B-1} B^{-(j+1)}. \tag{12}$$

Using $\rho = \frac{a}{B-1}$,

$$|\varepsilon_{j+1}| \le \rho B^{-(j+1)}. \tag{13}$$

Equivalently,

$$\left|\frac{N}{D} - Q_{j+1}\right| \leq \rho B^{-(j+1)}. \tag{14}$$

To obtain an "integer-like" quantity for hardware, define a scaled residual:

$$\left|B^{j+1}(N - DQ_{j+1})\right| \leq \rho D, \tag{15}$$

and set

$$R_{j+1} = B^{j+1}(N - DQ_{j+1}). \tag{16}$$

This yields the *containment condition*:

$$|R_{j+1}| \leq \rho D. \tag{17}$$

At the end of the algorithm, the final residual relates to the final remainder:

$$\text{At step } j = n-1, \quad R_n = B^n(N - DQ_n) = B^n r. \tag{18}$$

3.3 Residual recurrence

Starting from eq. (16) and using eq. (8),

$$\begin{aligned}
R_{j+1} &= B^{j+1}(N - DQ_{j+1}) \\
&= B^{j+1}\left(N - D\left(Q_j + q_{j+1}B^{-(j+1)}\right)\right) \\
&= B^{j+1}(N - DQ_j) - q_{j+1}D \\
&= B\left(B^j(N - DQ_j)\right) - q_{j+1}D \\
&= BR_j - q_{j+1}D.
\end{aligned}$$

Thus the central recurrence for division is

$$R_{j+1} = BR_j - q_{j+1}D. \tag{19}$$

Each iteration therefore consists of:

1. choosing a digit q_{j+1} from the redundant set, using (possibly truncated) information about R_j and D, and
2. updating the residual using eq. (19),

with the requirement that the next residual satisfy eq. (17).

3.4 Selection Intervals for a Positive Divisor

For a positive divisor $D \in \left[\frac{1}{2}, 1\right)$, the containment condition

$$|R_{j+1}| \leq \rho D$$

is equivalent to

$$-\rho D \leq R_{j+1} \leq \rho D. \tag{20}$$

Substitute eq. (19):

$$-\rho D \leq BR_j - q_{j+1}D \leq \rho D. \tag{21}$$

Adding $q_{j+1}D$ yields the selection interval for digit q_{j+1}:

$$(q_{j+1} - \rho)D \leq BR_j \leq (q_{j+1} + \rho)D. \tag{22}$$

These intervals overlap when $\rho > \frac{1}{2}$, which is the property that enables robust selection under truncation.

3.5 Selection Intervals for a Negative Divisor

For a negative divisor $D \in \left[-1, -\frac{1}{2}\right)$, $|D| = -D$. Containment can be written as

$$|R_{j+1}| \leq \rho|D| \implies |R_{j+1}| \leq -\rho D, \tag{23}$$

which expands to

$$\rho D \leq R_{j+1} \leq -\rho D. \tag{24}$$

Substitute eq. (19):

$$\rho D \leq BR_j - q_{j+1}D \leq -\rho D. \tag{25}$$

Adding $q_{j+1}D$ yields

$$(q_{j+1} + \rho)D \leq BR_j \leq (q_{j+1} - \rho)D, \tag{26}$$

i.e., the interval endpoints reverse because $D < 0$.

3.6 Initialization

The digit-recurrence update eq. (19) requires a starting value R_0. However, the scaled-residual definition

$$R_{j+1} = B^{j+1}(N - DQ_{j+1}), \qquad j = 0, 1, \ldots, n-1$$

defines $R_1, \ldots, R_n$ (not R_0). Therefore, R_0 must be specified as part of the initialization convention.

We use the common choice $Q_0 = 0$. To match the selection-range checks used below (the "first-shift equals dividend" presentation), we initialize the residual so that the *shifted* quantity used by the first digit selection is exactly the dividend:

$$BR_0 = N \qquad \iff \qquad R_0 = \frac{N}{B}. \tag{27}$$

With this convention, the first digit-selection step compares BR_0 (which equals N) against the selection bounds, and the first residual update is performed by the same recurrence as in later iterations:

$$R_1 = BR_0 - q_1 D.$$

With the assumed operand ranges and a positive divisor $D \in \left[\frac{1}{2}, 1\right)$, the dividend is $N \in \left[\frac{1}{4}, \frac{1}{2}\right)$, so the shifted initial residual BR_0 also lies in $\left[\frac{1}{4}, \frac{1}{2}\right)$.

To verify this initialization, we show that the range of BR_0 is fully covered by the union of the selection intervals for q_1 (using eq. (22) for a positive divisor). The selection condition for q_1 is:

$$(q_1 - \rho)D \leq BR_0 \leq (q_1 + \rho)D. \tag{28}$$

Since BR_0 is positive, it suffices to check non-negative digits ($q_1 \in \{0, 1, \ldots, a\}$).

In the Radix-2 case ($B = 2$, $\rho = 1$), the shifted residual range is $2R_0 = N \in \left[\frac{1}{4}, \frac{1}{2}\right)$. We check the selection intervals for $q_1 = 0$ and $q_1 = 1$:

- For $q_1 = 1$: The interval is $[(1-1)D, (1+1)D] = [0, 2D]$.
- For $q_1 = 0$: The interval is $[(0-1)D, (0+1)D] = [-D, D]$.

The union of these intervals is $[-D, 2D]$. For any $D \in \left[\frac{1}{2}, 1\right)$, this union covers at least $\left[-\frac{1}{2}, 1\right]$. This range safely contains the required range of $2R_0 \in \left[\frac{1}{4}, \frac{1}{2}\right)$, so the initialization is valid.

In the Radix-4 case ($B = 4$, $\rho = 2/3$), the shifted residual range is $4R_0 = N \in \left[\frac{1}{4}, \frac{1}{2}\right)$. We check the intervals for $q_1 \in \{0, 1, 2\}$:

- For $q_1 = 2$: The interval is $\left[\left(2 - \frac{2}{3}\right)D, \left(2 + \frac{2}{3}\right)D\right] = \left[\frac{4}{3}D, \frac{8}{3}D\right]$.

- For $q_1 = 1$: The interval is $\left[\left(1 - \frac{2}{3}\right) D, \left(1 + \frac{2}{3}\right) D\right] = \left[\frac{1}{3}D, \frac{5}{3}D\right]$.
- For $q_1 = 0$: The interval is $\left[\left(0 - \frac{2}{3}\right) D, \left(0 + \frac{2}{3}\right) D\right] = \left[-\frac{2}{3}D, \frac{2}{3}D\right]$.

The union of these intervals is $\left[-\frac{2}{3}D, \frac{8}{3}D\right]$. For any $D \in \left[\frac{1}{2}, 1\right)$, this union covers at least $\left[-\frac{1}{3}, \frac{4}{3}\right]$. This range safely contains the required range of $4R_0 \in \left[\frac{1}{4}, \frac{1}{2}\right)$, confirming the initialization is also valid for the Radix-4 case.

For a negative divisor, the analysis is analogous using the selection intervals from eq. (26). The principle of ensuring the initial shifted residual falls within the union of valid selection regions remains the same.

3.7 Example: Radix-2 SRT Division

Consider Radix-2 ($B = 2$) with a maximally redundant digit set $\{-1, 0, 1\}$, i.e., $a = 1$. Then

$$\rho = \frac{a}{B-1} = \frac{1}{2-1} = 1. \tag{29}$$

3.7.1 Positive Divisor

With $\rho = 1$ and $B = 2$, the selection intervals from eq. (22) become

$$q_{j+1} = \begin{cases} 1 & \text{if } 0 \le 2R_j \le 2D \\ 0 & \text{if } -D \le 2R_j \le D \\ -1 & \text{if } -2D \le 2R_j \le 0. \end{cases} \tag{30}$$

These intervals overlap (e.g., if $2R_j = 0.2D$, then both $q_{j+1} = 1$ and $q_{j+1} = 0$ are valid).

In hardware, R_j is often maintained in carry-save form to avoid full carry propagation in the main loop. Digit selection, however, needs a comparable quantity derived from $2R_j$. To keep selection fast, we use a truncated estimate based on only the most significant bits.

Let the full-precision shifted residual be $2R_j = r_{j,2\ldots0}.r_{j,-1\ldots-\infty}$, represented as the sum of two carry-save vectors: $Y_j = y_{j,2\ldots0}.y_{j,-1\ldots-\infty}$ and $Z_j = z_{j,2\ldots0}.z_{j,-1\ldots-\infty}$:

$$r_{j,2\ldots0}.r_{j,-1\ldots-\infty} = y_{j,2\ldots0}.y_{j,-1\ldots-\infty} + z_{j,2\ldots0}.z_{j,-1\ldots-\infty}. \tag{31}$$

Form a truncated estimate t_j by adding only the top bits of Y_j and Z_j using a small CPA. Using α fractional bits:

$$t_{j,2\ldots0}.t_{j,-1\ldots-\alpha} = y_{j,2\ldots0}.y_{j,-1\ldots-\alpha} + z_{j,2\ldots0}.z_{j,-1\ldots-\alpha}. \tag{32}$$

This truncation introduces a one-sided error:

$$0 \le 2R_j - t_{j,2\ldots0}.t_{j,-1\ldots-\alpha} < 2 \cdot 2^{-\alpha} = 2^{-(\alpha-1)}. \tag{33}$$

Similarly, use a truncated divisor D with its first β fractional bits. Define integer-valued estimates for table-based selection (PLA):

$$\tau_j = \mathrm{Int}(2^\alpha \cdot t_{j,2\ldots0}.t_{j,-1\ldots-\alpha}) \in \left\{-2^{\alpha+2}, \ldots, 2^{\alpha+2} - 1\right\} \qquad (\tau_j \le 2^{\alpha+1} R_j < \tau_j + 2), \tag{34}$$

$$\delta = \mathrm{Int}(2^\beta \cdot 0.1d_{j,-2\ldots-\beta}) \in \left\{2^{\beta-1}, \ldots, 2^\beta - 1\right\} \qquad (\delta \le 2^\beta D < \delta + 1). \tag{35}$$

The selector chooses $k \in \{-1, 0, 1\}$ based on (τ_j, δ).

To guarantee correct selection with truncated estimates, overlap must absorb truncation uncertainty. This is captured by the *continuity condition*. Let $L_k(D) = (k - \rho)D$ and $U_k(D) = (k + \rho)D$ denote the ideal lower and upper bounds for digit k. The condition requires that adjacent regions overlap even under worst-case truncation:

$$\min(U_{k-1,\mathrm{trunc}}) \ge \max(L_{k,\mathrm{trunc}}). \tag{36}$$

Truncation of the residual lowers the effective upper bound by up to $2^{-(\alpha-1)}$. Truncation of the divisor yields $D \in \left[2^{-\beta}\delta, 2^{-\beta}(\delta+1)\right)$. Assuming positive coefficients of D:

$$\min(U_{k-1,\text{trunc}}) = \min_D(U_{k-1}(D)) - 2^{-(\alpha-1)} = (k-1+\rho)(2^{-\beta}\delta) - 2^{-(\alpha-1)}, \tag{37}$$

$$\max(L_{k,\text{trunc}}) = \max_D(L_k(D)) = (k-\rho)(2^{-\beta}(\delta+1)). \tag{38}$$

For negative coefficients, the roles of δ and $\delta+1$ are swapped.
For the boundaries between regions:

- For $k=1$ (U_0, L_1): $(0+1)(2^{-\beta}\delta) - 2^{-(\alpha-1)} \geq (1-1)(2^{-\beta}(\delta+1))$, which simplifies to $2^{-\beta}\delta - 2^{-(\alpha-1)} \geq 0$.
- For $k=0$ (U_{-1}, L_0): $(-1+1)(2^{-\beta}(\delta+1)) - 2^{-(\alpha-1)} \geq (0-1)(2^{-\beta}\delta)$, which simplifies to $2^{-\beta}\delta \geq 2^{-(\alpha-1)}$.

The condition must hold for the worst case (smallest δ). Since $D \in [1/2, 1)$, $\delta_{\min} = 2^{\beta-1}$. Thus

$$2^{-\beta}(2^{\beta-1}) \geq 2^{-(\alpha-1)} \;\Rightarrow\; \alpha \geq 2.$$

Choosing $\alpha = 2$, $\beta = 2$:

$$\tau_j = \text{Int}(4 \cdot t_{j,2\ldots0}.t_{j,-1}t_{j,-2}) \in \{-16, \ldots, 15\} \qquad (\tau_j \leq 8R_j < \tau_j + 2), \tag{39}$$

$$\delta = \text{Int}(4 \cdot 0.1d_{j,-2}) \in \{2, 3\} \qquad (\delta \leq 4D < \delta + 1). \tag{40}$$

The final selection logic is the table

$$q_{j+1} = \begin{cases} 1 & \text{if } 0 \leq \tau_j < 2(\delta+1) \\ 0 & \text{if } -\delta \leq \tau_j \leq \delta - 2 \\ -1 & \text{if } -2(\delta+1) - 2 < \tau_j \leq -2. \end{cases} \tag{41}$$

3.7.2 Negative Divisor

For $D \in [-1, -1/2)$, selection intervals invert:

$$q_{j+1} = \begin{cases} 1 & \text{if } 2D \leq 2R_j \leq 0 \\ 0 & \text{if } D \leq 2R_j \leq -D \\ -1 & \text{if } 0 \leq 2R_j \leq -2D. \end{cases} \tag{42}$$

The continuity condition becomes $\min(U_{k,\text{trunc}}) \geq \max(L_{k-1,\text{trunc}})$, where $U_k = (k-\rho)D$ and $L_k = (k+\rho)D$. Care is required because multiplying inequalities by negative numbers reverses order. The same precision requirement $\alpha \geq 2$ results. With $\alpha = 2$, $\beta = 2$:

$$\tau_j = \text{Int}(4 \cdot t_{j,2\ldots0}.t_{j,-1}t_{j,-2}) \in \{-16, \ldots, 15\} \qquad (\tau_j \leq 8R_j < \tau_j + 2), \tag{43}$$

$$\delta = \text{Int}(4 \cdot 1.0d_{j,-2}) \in \{-4, -3\} \qquad (\delta \leq 4D < \delta + 1). \tag{44}$$

The corresponding selection logic is

$$q_{j+1} = \begin{cases} 1 & \text{if } 2\delta - 2 < \tau_j \leq -2 \\ 0 & \text{if } \delta + 1 \leq \tau_j \leq -(\delta+1) - 2 \\ -1 & \text{if } 0 \leq \tau_j \leq -2\delta. \end{cases} \tag{45}$$

3.8 Example: Radix-4 SRT Division

For Radix-4 ($B = 4$), a common digit set is $\{-2, -1, 0, 1, 2\}$ so $a = 2$ and

$$\rho = \frac{a}{B-1} = \frac{2}{4-1} = \frac{2}{3}. \tag{46}$$

3.8.1 Positive Divisor

From eq. (22), the selection intervals are

$$q_{j+1} = \begin{cases} 2 & \text{if } \frac{4}{3}D \le 4R_j \le \frac{8}{3}D \\ 1 & \text{if } \frac{1}{3}D \le 4R_j \le \frac{5}{3}D \\ 0 & \text{if } -\frac{2}{3}D \le 4R_j \le \frac{2}{3}D \\ -1 & \text{if } -\frac{5}{3}D \le 4R_j \le -\frac{1}{3}D \\ -2 & \text{if } -\frac{8}{3}D \le 4R_j \le -\frac{4}{3}D. \end{cases} \tag{47}$$

As in Radix-2, the residual is held in carry-save form and a truncated estimate t_j is computed using α fractional bits. Truncation yields $0 \le 4R_j - t_j < 2^{-(\alpha-1)}$. Define integer estimates:

$$\tau_j = \mathrm{Int}(2^{\alpha} \cdot t_{j,2\ldots0}.t_{j,-1\ldots-\alpha}) \in \left\{-2^{\alpha+2}, \ldots, 2^{\alpha+2}-1\right\} \qquad (\tau_j \le 2^{\alpha+2}R_j < \tau_j + 2), \tag{48}$$

$$\delta = \mathrm{Int}(2^{\beta} \cdot 0.1d_{j,-2\ldots-\beta}) \in \left\{2^{\beta-1}, \ldots, 2^{\beta}-1\right\} \qquad (\delta \le 2^{\beta}D < \delta + 1). \tag{49}$$

The continuity condition $\min(U_{k-1,\mathrm{trunc}}) \ge \max(L_{k,\mathrm{trunc}})$ must hold for $k \in \{-1, 0, 1, 2\}$. Substituting $\rho = 2/3$ and simplifying yields:

- For $k = 2$: $\frac{1}{3} \cdot 2^{-\beta}\delta - \frac{4}{3} \cdot 2^{-\beta} - 2^{-(\alpha-1)} \ge 0$.
- For $k = 1$: $\frac{1}{3} \cdot 2^{-\beta}\delta - \frac{1}{3} \cdot 2^{-\beta} - 2^{-(\alpha-1)} \ge 0$.
- For $k = 0$: $\frac{1}{3} \cdot 2^{-\beta}\delta - \frac{1}{3} \cdot 2^{-\beta} - 2^{-(\alpha-1)} \ge 0$.
- For $k = -1$: $\frac{1}{3} \cdot 2^{-\beta}\delta - \frac{4}{3} \cdot 2^{-\beta} - 2^{-(\alpha-1)} \ge 0$.

The most restrictive conditions are for $k = 2$ and $k = -1$. A common choice is $\alpha = 5, \beta = 4$:

$$\tau_j = \mathrm{Int}(32 \cdot t_{j,2\ldots0}.t_{j,-1\ldots-5}) \in \{-128, \ldots, 127\} \qquad (\tau_j \le 128R_j < \tau_j + 2), \tag{50}$$

$$\delta = \mathrm{Int}(16 \cdot 0.1d_{j,-2\ldots-4}) \in \{8, \ldots, 15\} \qquad (\delta \le 16D < \delta + 1). \tag{51}$$

The resulting selection logic is

$$q_{j+1} = \begin{cases} 2 & \text{if } \frac{8}{3}(\delta+1) \le \tau_j < \frac{16}{3}(\delta+1) \\ 1 & \text{if } \frac{2}{3}(\delta+1) \le \tau_j \le \frac{10}{3}\delta - 2 \\ 0 & \text{if } -\frac{4}{3}\delta \le \tau_j \le \frac{4}{3}\delta - 2 \\ -1 & \text{if } -\frac{10}{3}\delta \le \tau_j \le -\frac{2}{3}(\delta+1) - 2 \\ -2 & \text{if } -\frac{16}{3}(\delta+1) - 2 < \tau_j \le -\frac{8}{3}(\delta+1) - 2. \end{cases} \tag{52}$$

3.8.2 Negative Divisor

For $D \in \left[-1, -\frac{1}{2}\right)$, intervals reverse:

$$q_{j+1} = \begin{cases} 2 & \text{if } \frac{8}{3}D \le 4R_j \le \frac{4}{3}D \\ 1 & \text{if } \frac{5}{3}D \le 4R_j \le \frac{1}{3}D \\ 0 & \text{if } \frac{2}{3}D \le 4R_j \le -\frac{2}{3}D \\ -1 & \text{if } -\frac{1}{3}D \le 4R_j \le -\frac{5}{3}D \\ -2 & \text{if } -\frac{4}{3}D \le 4R_j \le -\frac{8}{3}D. \end{cases} \tag{53}$$

The analysis proceeds analogously using $\min(U_{k,\mathrm{trunc}}) \ge \max(L_{k-1,\mathrm{trunc}})$. The same precision parameters ($\alpha = 5, \beta = 4$) are sufficient:

$$\tau_j = \mathrm{Int}(32 \cdot t_{j,2\ldots0}.t_{j,-1\ldots-5}) \in \{-128, \ldots, 127\} \qquad (\tau_j \le 128R_j < \tau_j + 2), \tag{54}$$

$$\delta = \mathrm{Int}(16 \cdot 1.0d_{j,-2\ldots-4}) \in \{-16, \ldots, -9\} \qquad (\delta \le 16D < \delta + 1). \tag{55}$$

The selection logic is

$$q_{j+1} = \begin{cases} 2 & \text{if } \frac{16}{3}\delta - 2 < \tau_j \le \frac{8}{3}\delta - 2 \\ 1 & \text{if } \frac{10}{3}(\delta+1) \le \tau_j \le \frac{2}{3}\delta - 2 \\ 0 & \text{if } \frac{4}{3}(\delta+1) \le \tau_j \le -\frac{4}{3}(\delta+1) - 2 \\ -1 & \text{if } -\frac{2}{3}\delta \le \tau_j \le -\frac{10}{3}(\delta+1) - 2 \\ -2 & \text{if } -\frac{8}{3}\delta \le \tau_j \le -\frac{16}{3}\delta. \end{cases} \tag{56}$$

4 Square Root by Digit Recurrence

Digit recurrence can also be used for square root extraction. The fundamental relationship is

$$X = S_n^2 + r, \tag{57}$$

where X is the radicand, S_n the final root, and r the final (unscaled) remainder.
We normalize

$$X \in \left[\frac{1}{4}, 1\right) \qquad \text{(Radicand)}$$

$$S_n \in \left[\sqrt{\frac{1}{4}}, \sqrt{1}\right) = \left[\frac{1}{2}, 1\right) \qquad \text{(Square Root).}$$

4.1 Root digits and approximation

The root is generated over n iterations with $j = 0, 1, \ldots, n-1$. Let S_j be the approximation after j digits $(s_1, \ldots, s_j)$.

$$S_{j+1} = S_j + s_{j+1}B^{-(j+1)}. \tag{58}$$

Unfolding,

$$\text{At step } j, \quad S_{j+1} = S_0 + \sum_{i=0}^{j} s_{i+1}B^{-(i+1)}. \tag{59}$$

After n iterations,

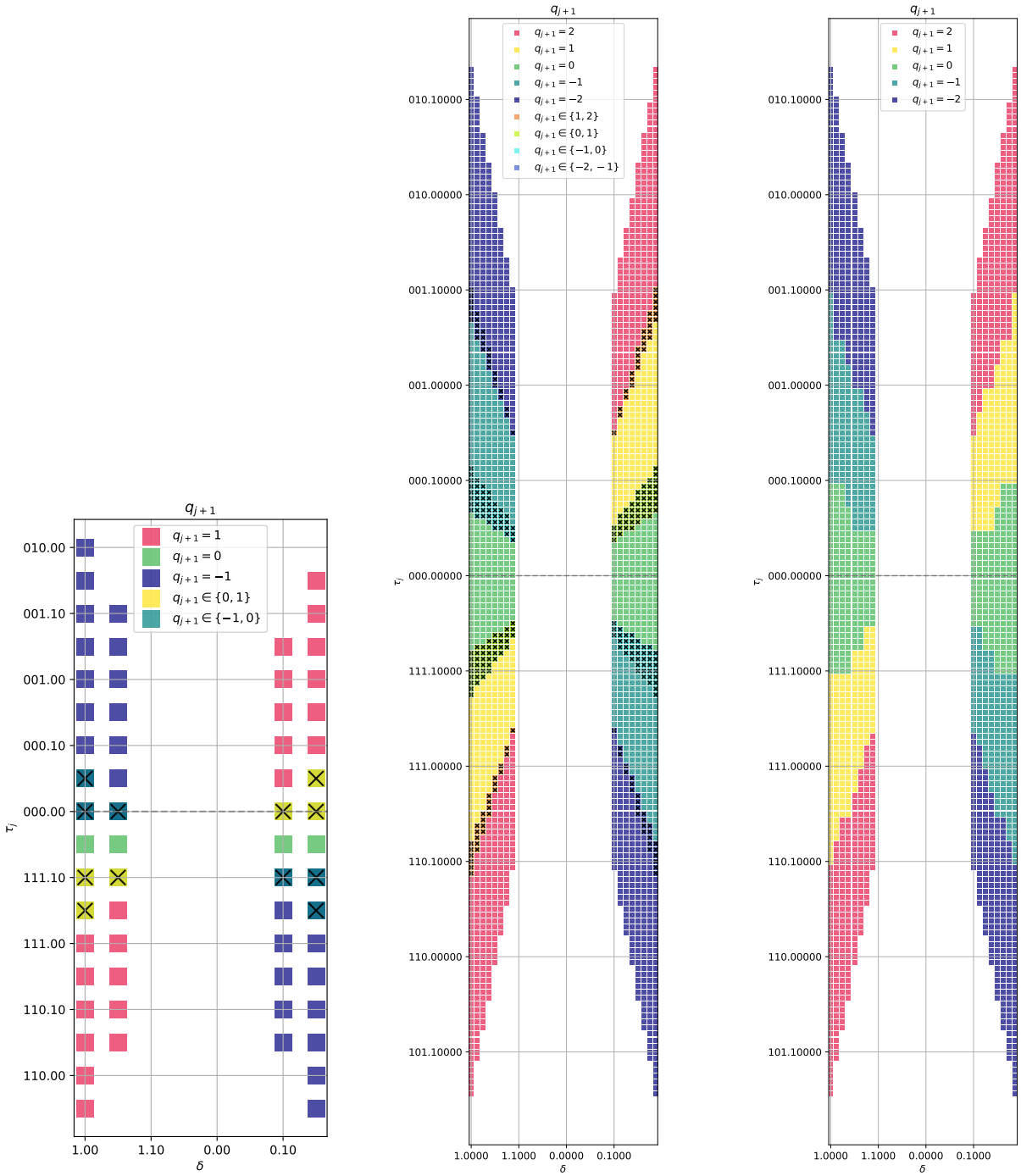
$$\text{At step } j = n-1, \quad S_n = S_0 + \sum_{i=0}^{n-1} s_{i+1}B^{-(i+1)}. \tag{60}$$

4.2 Error bound and containment

Define the error $\varepsilon_{j+1} = \sqrt{X} - S_{j+1}$. As in division,

$$\varepsilon_{j+1} = \sum_{i=j+1}^{\infty} s_{i+1}B^{-(i+1)}, \tag{61}$$

and with $|s_{i+1}| \le a$,



(a) Radix-2 Basic QDS

(b) Radix-4 Basic QDS

(c) Radix-4 Optimized QDS

Figure 1: Selection regions for the Quotient Digit Selector (QDS). The y-axis is the truncated residual estimate τ_j and the x-axis is the truncated divisor estimate δ. The plots show how the choice of quotient digit q_{j+1} depends on these estimates.

$$\left|\sum_{i=j+1}^{\infty} s_{i+1}B^{-(i+1)}\right| \leq \sum_{i=j+1}^{\infty} aB^{-(i+1)} = aB^{-(j+2)}\sum_{k=0}^{\infty} B^{-k} = aB^{-(j+2)}\frac{1}{1-B^{-1}} = \frac{a}{B-1}B^{-(j+1)}. \tag{62}$$

Thus

$$|\varepsilon_{j+1}| \leq \rho B^{-(j+1)} \implies \left|\sqrt{X} - S_{j+1}\right| \leq \rho B^{-(j+1)}. \tag{63}$$

Rearrange and square:

$$S_{j+1} - \rho B^{-(j+1)} \leq \sqrt{X} \leq S_{j+1} + \rho B^{-(j+1)}, \tag{64}$$

$$S_{j+1}^2 - 2S_{j+1}\rho B^{-(j+1)} + \rho^2 B^{-2(j+1)} \leq X \leq S_{j+1}^2 + 2S_{j+1}\rho B^{-(j+1)} + \rho^2 B^{-2(j+1)}. \tag{65}$$

Isolate $X - S_{j+1}^2$:

$$\rho^2 B^{-2(j+1)} - 2S_{j+1}\rho B^{-(j+1)} \leq X - S_{j+1}^2 \leq \rho^2 B^{-2(j+1)} + 2S_{j+1}\rho B^{-(j+1)}. \tag{66}$$

Define the scaled residual

$$R_{j+1} = B^{j+1}(X - S_{j+1}^2).$$

Multiplying eq. (66) by B^{j+1} yields

$$\rho^2 B^{-(j+1)} - 2\rho S_{j+1} \leq B^{j+1}(X - S_{j+1}^2) \leq \rho^2 B^{-(j+1)} + 2\rho S_{j+1}, \tag{67}$$

i.e.,

$$\rho^2 B^{-(j+1)} - 2\rho S_{j+1} \leq R_{j+1} \leq \rho^2 B^{-(j+1)} + 2\rho S_{j+1}. \tag{68}$$

At the end,

$$\text{At step } j = n-1, \quad R_n = B^n(X - S_n^2) = B^n r. \tag{69}$$

4.3 Residual recurrence

From the definition,

$$\begin{aligned}
R_{j+1} &= B^{j+1}(X - S_{j+1}^2) \\
&= B^{j+1}\left(X - \left(S_j + s_{j+1}B^{-(j+1)}\right)^2\right) \\
&= B^{j+1}\left(X - \left(S_j^2 + 2S_j s_{j+1}B^{-(j+1)} + s_{j+1}^2 B^{-2(j+1)}\right)\right) \\
&= B^{j+1}(X - S_j^2) - B^{j+1}\left(2S_j s_{j+1}B^{-(j+1)} + s_{j+1}^2 B^{-2(j+1)}\right) \\
&= B\left(B^j(X - S_j^2)\right) - \left(2S_j s_{j+1} + s_{j+1}^2 B^{-(j+1)}\right) \\
&= BR_j - s_{j+1}\left(2S_j + s_{j+1}B^{-(j+1)}\right).
\end{aligned}$$

Thus

$$R_{j+1} = BR_j - s_{j+1}\left(2S_j + s_{j+1}B^{-(j+1)}\right). \tag{70}$$

4.4 Selection intervals

Substitute eq. (70) into eq. (68):

$$\rho^2 B^{-(j+1)} - 2\rho S_{j+1} \le BR_j - s_{j+1}\left(2S_j + s_{j+1}B^{-(j+1)}\right) \le \rho^2 B^{-(j+1)} + 2\rho S_{j+1}. \tag{71}$$

Using $S_{j+1} = S_j + s_{j+1}B^{-(j+1)}$:

$$\rho^2 B^{-(j+1)} - 2\rho\left(S_j + s_{j+1}B^{-(j+1)}\right) \le BR_j - s_{j+1}\left(2S_j + s_{j+1}B^{-(j+1)}\right) \le \rho^2 B^{-(j+1)} + 2\rho\left(S_j + s_{j+1}B^{-(j+1)}\right). \tag{72}$$

Rearranging yields the selection interval for BR_j:

$$2(s_{j+1} - \rho)S_j + (s_{j+1} - \rho)^2 B^{-(j+1)} \le BR_j \le 2(s_{j+1} + \rho)S_j + (s_{j+1} + \rho)^2 B^{-(j+1)}. \tag{73}$$

Unlike division, these bounds depend on S_j, which changes each iteration; this is the main reason square-root digit selection is more involved.

4.5 Initialization

The first iteration ($j = 0$) needs S_0 and an initial residual R_0 such that BR_0 lies in a valid selection region for s_1. However, the scaled-residual relation

$$R_{j+1} = B^{j+1}(X - S_{j+1}^2), \qquad j = 0, 1, \ldots, n-1$$

defines $R_1, \ldots, R_n$ (not R_0), so R_0 must be specified as part of the initialization convention.
A common and robust initialization strategy sets $S_0 = 1$ and initializes the residual by convention as

$$R_0 = X - 1. \tag{74}$$

Since $X \in [1/4, 1)$, $R_0 \in [-3/4, 0)$.
To verify initialization, substitute $j = 0$ and $S_0 = 1$ into eq. (73):

$$2(s_1 - \rho) + (s_1 - \rho)^2 B^{-1} \le BR_0 \le 2(s_1 + \rho) + (s_1 + \rho)^2 B^{-1}. \tag{75}$$

Since BR_0 is negative, it suffices to check non-positive digits ($s_1 \in \{0, -1, \ldots, -a\}$).
In the Radix-2 case ($B = 2$, $\rho = 1$), $2R_0 \in \left[-\frac{3}{2}, 0\right)$. We check $s_1 = 0$ and $s_1 = -1$:

- For $s_1 = 0$: The interval is $[2(0-1) + (0-1)^2 2^{-1}, 2(0+1) + (0+1)^2 2^{-1}] = [-1.5, 2.5]$.
- For $s_1 = -1$: The interval is $[2(-1-1) + (-1-1)^2 2^{-1}, 2(-1+1) + (-1+1)^2 2^{-1}] = [-2, 0]$.

Their union covers $[-2, 2.5]$, which contains $2R_0 \in [-1.5, 0)$.
In the Radix-4 case ($B = 4$, $\rho = 2/3$), $4R_0 \in [-3, 0)$. We check $s_1 \in \{0, -1, -2\}$:

- For $s_1 = 0$: The interval is $\left[-\frac{4}{3} + \frac{1}{9}, \frac{4}{3} + \frac{1}{9}\right] \approx [-1.22, 1.44]$.
- For $s_1 = -1$: The interval is $\left[-\frac{10}{3} + \frac{25}{36}, -\frac{2}{3} + \frac{1}{36}\right] \approx [-2.64, -0.64]$.
- For $s_1 = -2$: The interval is $\left[-\frac{16}{3} + \frac{16}{9}, -\frac{8}{3} + \frac{4}{9}\right] \approx [-3.56, -2.22]$.

Their union covers $[-3.56, 1.44]$, which contains $4R_0 \in [-3, 0)$.

4.6 Example: Radix-2 SRT Square Root

Use $B = 2$, $a = 1$, so $\rho = 1$ and $s_{j+1} \in \{-1, 0, 1\}$. From eq. (73):

$$s_{j+1} = \begin{cases} 1 & \text{if } 0 \le 2R_j \le 4S_j + 4 \cdot 2^{-(j+1)} \\ 0 & \text{if } -2S_j + 2^{-(j+1)} \le 2R_j \le 2S_j + 2^{-(j+1)} \\ -1 & \text{if } -4S_j + 4 \cdot 2^{-(j+1)} \le 2R_j \le 0. \end{cases} \tag{76}$$

Selection depends on both $2R_j$ and S_j, so we need truncated estimates of both. The residual estimate t_j has error $0 \le 2R_j - t_j < 2^{-(\alpha-1)}$. Define

$$\tau_j = \mathrm{Int}(2^\alpha \cdot t_{j,3\ldots0}.t_{j,-1\ldots-\alpha}) \in \left\{-2^{\alpha+3}, \ldots, 2^{\alpha+3} - 1\right\} \qquad (\tau_j \le 2^{\alpha+1} R_j < \tau_j + 2), \tag{77}$$

$$\sigma_j = \mathrm{Int}(2^\beta \cdot 0.1 s_{j,-2\ldots-\beta}) \in \left\{2^{\beta-1}, \ldots, 2^\beta - 1\right\} \qquad (\sigma_j \le 2^\beta S_j \le \sigma_j + 1 - 2^\beta \mathrm{ulp}). \tag{78}$$

Here $S_j \in [1/2, 1)$ and $\mathrm{ulp} = 2^{-j}$, so $\sigma_j \le 2^\beta S_j \le \sigma_j + 1 - 2^\beta 2^{-j}$.
Continuity analysis uses $\min(U_{k-1,\mathrm{trunc}}) \ge \max(L_{k,\mathrm{trunc}})$ with $\rho = 1$, $B = 2$:

- For $k = 1$: $\min(U_{0,\mathrm{trunc}}) \ge \max(L_{1,\mathrm{trunc}})$ gives $2 \cdot 2^{-\beta}\sigma_j - 2^{-(\alpha-1)} \ge 0$. This holds for $\alpha \ge 1$ since $\sigma_j \ge 2^{\beta-1}$.
- For $k = 0$: $\min(U_{-1,\mathrm{trunc}}) \ge \max(L_{0,\mathrm{trunc}})$ gives $2 \cdot 2^{-\beta}\sigma_j - 2^{-(\alpha-1)} - B^{-(j+1)} \ge 0$.

For worst case $\sigma_j = 2^{\beta-1}$:

$$1 - 2^{-(\alpha-1)} - 2^{-(j+1)} \ge 0 \;\Rightarrow\; \alpha \ge 2.$$

Choose $\alpha = 2$, $\beta = 2$:

$$\tau_j = \mathrm{Int}(4 \cdot t_{j,3\ldots0}.t_{j,-1}t_{j,-2}) \in \{-32, \ldots, 31\} \qquad (\tau_j \le 8R_j < \tau_j + 2), \tag{79}$$

$$\sigma_j = \mathrm{Int}(4 \cdot 0.1 s_{j,-2}) \in \{2, 3\} \qquad (\sigma_j \le 4S_j \le \sigma_j + 1 - 4 \cdot 2^{-j}). \tag{80}$$

Selection logic:

$$s_{j+1} = \begin{cases} 1 & \text{if } 0 \le \tau_j < 4(\sigma_j + 1) \\ 0 & \text{if } -2(\sigma_j - 1) \le \tau_j \le 2\sigma_j - 2 \\ -1 & \text{if } -4(\sigma_j + 1) - 2 < \tau_j \le -2. \end{cases} \tag{81}$$

4.6.1 Special Case: Radicand Close to 1

A special condition can occur in early iterations if the first selected digits are zero. If $s_1, \ldots, s_j = 0$, then $S_j = S_0 = 1$, so the root estimate is constant and σ_j becomes constant as well. This case is particularly relevant for X close to 1.

For Radix-2 with $\alpha = 2$, $\beta = 2$, τ_j estimates the scaled residual $2R_j$. Since $R_0 = X - 1 < 0$ and $R_k = BR_{k-1}$ when $s_k = 0$, the residual remains negative as long as only zeros are selected; thus $\tau_j < 0$. With $S_j = 1$:

$$\tau_j = \mathrm{Int}(4 \cdot 1 t_{j,2\ldots0}.t_{j,-1}t_{j,-2}) \in \{-32, \ldots, -1\} \qquad (\tau_j \le 8R_j < \tau_j + 2), \tag{82}$$

$$\sigma_j = \mathrm{Int}(4 \cdot 1.00) = 4 \qquad (4S_j = \sigma_j). \tag{83}$$

Selection reduces to

$$s_{j+1} = \begin{cases} 0 & \text{if } -2(\sigma_j - 1) \le \tau_j \le -1 \\ -1 & \text{if } -4\sigma_j - 2 < \tau_j \le -2. \end{cases} \tag{84}$$

4.7 Example: Radix-4 SRT Square Root

Use $B = 4$, $a = 2$, so $\rho = 2/3$ and $s_{j+1} \in \{-2, -1, 0, 1, 2\}$. From eq. (73):

$$s_{j+1} = \begin{cases} 2 & \text{if } \frac{8}{3}S_j + \frac{16}{9}4^{-(j+1)} \le 4R_j \le \frac{16}{3}S_j + \frac{64}{9}4^{-(j+1)} \\ 1 & \text{if } \frac{2}{3}S_j + \frac{1}{9}4^{-(j+1)} \le 4R_j \le \frac{10}{3}S_j + \frac{25}{9}4^{-(j+1)} \\ 0 & \text{if } -\frac{4}{3}S_j + \frac{4}{9}4^{-(j+1)} \le 4R_j \le \frac{4}{3}S_j + \frac{4}{9}4^{-(j+1)} \\ -1 & \text{if } -\frac{10}{3}S_j + \frac{25}{9}4^{-(j+1)} \le 4R_j \le -\frac{2}{3}S_j + \frac{1}{9}4^{-(j+1)} \\ -2 & \text{if } -\frac{16}{3}S_j + \frac{64}{9}4^{-(j+1)} \le 4R_j \le -\frac{8}{3}S_j + \frac{16}{9}4^{-(j+1)}. \end{cases} \tag{85}$$

Define truncated estimates

$$\tau_j = \mathrm{Int}(2^\alpha \cdot t_{j,3\ldots0}.t_{j,-1\ldots-\alpha}) \in \left\{-2^{\alpha+3}, \ldots, 2^{\alpha+3} - 1\right\} \qquad (\tau_j \le 2^{\alpha+2}R_j < \tau_j + 2), \tag{86}$$

$$\sigma_j = \mathrm{Int}(2^\beta \cdot 0.1s_{j,-2\ldots-\beta}) \in \left\{2^{\beta-1}, \ldots, 2^\beta - 1\right\} \qquad (\sigma_j \le 2^\beta S_j \le \sigma_j + 1 - 2^\beta 4^{-j}). \tag{87}$$

Continuity must hold for $k \in \{-1, 0, 1, 2\}$. Substituting $\rho = 2/3$ gives

- For $k = 2$: $\frac{2}{3} \cdot 2^{-\beta}\sigma_j - \frac{8}{3} \cdot 2^{-\beta} - 2^{-(\alpha-1)} \ge 0$.
- For $k = 1$: $\frac{2}{3} \cdot 2^{-\beta}\sigma_j - \frac{2}{3} \cdot 2^{-\beta} - 2^{-(\alpha-1)} \ge 0$.
- For $k = 0$: $\frac{2}{3} \cdot 2^{-\beta}\sigma_j - \frac{2}{3} \cdot 2^{-\beta} - 2^{-(\alpha-1)} - \frac{4}{9}B^{-(j+1)} \ge 0$.
- For $k = -1$: $\frac{2}{3} \cdot 2^{-\beta}\sigma_j - \frac{8}{3} \cdot 2^{-\beta} - 2^{-(\alpha-1)} - \frac{25}{9}B^{-(j+1)} \ge 0$.

A key complication (not present in division) is the explicit $B^{-(j+1)}$ dependence in the bounds. This makes overlap iteration-dependent and most restrictive for small j and for negative digits with larger squared terms. The tightest boundary is between selecting $k = -1$ and $k = -2$:

$$\frac{2}{3} \cdot 2^{-\beta}\sigma_j - \frac{8}{3} \cdot 2^{-\beta} - 2^{-(\alpha-1)} - \frac{25}{9}B^{-(j+1)} \ge 0.$$

For example, with $\alpha = 4$, $\beta = 4$, $B = 4$, and worst-case $\sigma_j = 8$, the constant part is $1/24$ and the condition becomes

$$\frac{1}{24} - \frac{25}{9}4^{-(j+1)} \ge 0,$$

which holds only for $j \ge 3$. This failure for $j = 0, 1, 2$ indicates the need for a special startup mechanism (or more precision) in the earliest iterations.

For $j \ge 3$, use $\alpha = 4$, $\beta = 4$:

$$\tau_j = \mathrm{Int}(16 \cdot t_{j,3\ldots0}.t_{j,-1\ldots-4}) \in \{-128, \ldots, 127\} \qquad (\tau_j \le 64R_j < \tau_j + 2), \tag{88}$$

$$\sigma_j = \mathrm{Int}(16 \cdot 0.1s_{j,-2\ldots-4}) \in \{8, \ldots, 15\} \qquad (\sigma_j \le 16S_j \le \sigma_j + 1 - 16 \cdot 4^{-j}). \tag{89}$$

Selection logic for $j \ge 3$:

$$s_{j+1} = \begin{cases} 2 & \text{if } \frac{8}{3}(\sigma_j + 1) \le \tau_j < \frac{16}{3}(\sigma_j + 1) \\ 1 & \text{if } \frac{2}{3}(\sigma_j + 1) \le \tau_j \le \frac{10}{3}\sigma_j - 2 \\ 0 & \text{if } -\frac{4}{3}\left(\sigma_j - \frac{1}{48}\right) \le \tau_j \le \frac{4}{3}\sigma_j - 2 \\ -1 & \text{if } -\frac{10}{3}\left(\sigma_j - \frac{5}{96}\right) \le \tau_j \le -\frac{2}{3}(\sigma_j + 1) - 2 \\ -2 & \text{if } -\frac{16}{3}(\sigma_j + 1) - 2 < \tau_j \le -\frac{8}{3}(\sigma_j + 1) - 2. \end{cases} \tag{90}$$

4.7.1 Startup Issues and Iteration-Dependent Selection

As with Radix-2, consider the special case $s_1, \ldots, s_j = 0$, so $S_j = 1$ and σ_j is constant. However, Radix-4 shows that selection validity can depend on iteration index j.

The most restrictive continuity condition (between selecting -1 and -2) is

$$\frac{2}{3} \cdot 2^{-\beta}\sigma_j - \frac{8}{3} \cdot 2^{-\beta} - 2^{-(\alpha-1)} - \frac{25}{9}B^{-(j+1)} \geq 0.$$

Test it for $B = 4$, $\alpha = 4$, $\beta = 4$, and $\sigma_j = 16$:

$$\begin{aligned} \frac{2}{3} \cdot 2^{-4}(16) - \frac{8}{3} \cdot 2^{-4} - 2^{-(4-1)} - \frac{25}{9}4^{-(j+1)} &\geq 0 \\ \frac{2}{3} - \frac{1}{6} - \frac{1}{8} - \frac{25}{9}4^{-(j+1)} &\geq 0 \\ \frac{3}{8} - \frac{25}{9}4^{-(j+1)} &\geq 0. \end{aligned}$$

This depends on j:

- For $j = 0$: $\frac{3}{8} - \frac{25}{9 \cdot 4} = \frac{3}{8} - \frac{25}{36} = \frac{27-50}{72} = -\frac{23}{72} \ngeq 0$ (fails).
- For $j = 1$: $\frac{3}{8} - \frac{25}{9 \cdot 16} = \frac{3}{8} - \frac{25}{144} = \frac{54-25}{144} = \frac{29}{144} \geq 0$ (holds).

Thus the standard selection logic is not guaranteed to work in the very first iteration, requiring a special startup mechanism.

For $j \geq 1$, with $\alpha = 4$, $\beta = 4$:

$$\tau_j = \mathrm{Int}(16 \cdot 1t_{j,2\ldots0}.t_{j,-1\ldots-4}) \in \{-128, \ldots, -1\} \qquad (\tau_j \leq 64R_j < \tau_j + 2), \tag{91}$$

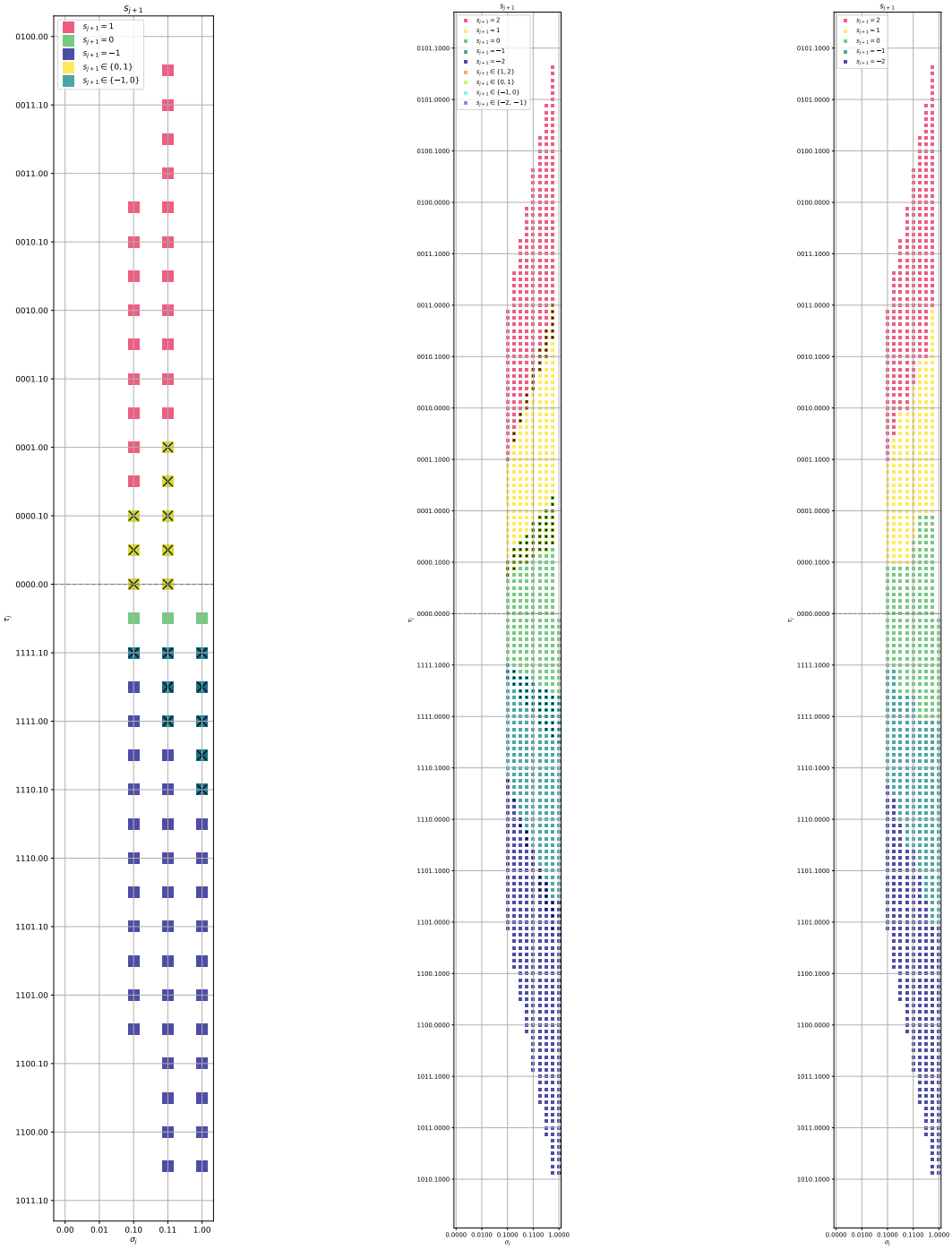
$$\sigma_j = \mathrm{Int}(16 \cdot 1.0000) = 16 \qquad (16S_j = \sigma_j). \tag{92}$$

Selection logic for $j \geq 1$:

$$s_{j+1} = \begin{cases} 0 & \text{if } -\frac{4}{3}\left(\sigma_j - \frac{1}{3}\right) \leq \tau_j \leq -1 \\ -1 & \text{if } -\frac{10}{3}\left(\sigma_j - \frac{5}{6}\right) \leq \tau_j \leq -\frac{2}{3}\sigma_j - 2 \\ -2 & \text{if } -\frac{16}{3}\sigma_j - 2 < \tau_j \leq -\frac{8}{3}\sigma_j - 2. \end{cases} \tag{93}$$

Table 1: Example selection logic constants for a Radix-4 implementation. The table shows the required integer range for the truncated residual estimate τ_j to select a given digit, based on the truncated divisor/root estimate (δ or σ_j).

τ_j	$q_{j+1}, s_{j+1} = 2$	$q_{j+1}, s_{j+1} = 1$	$q_{j+1}, s_{j+1} = 0$	$q_{j+1}, s_{j+1} = -1$	$q_{j+1}, s_{j+1} = -2$
$\delta, \sigma_j = 8$	24 to 49	8 to 23	-8 to 7	-26 to -9	-50 to -27
$\delta, \sigma_j = 9$	28 to 53	8 to 27	-8 to 7	-28 to -9	-56 to -29
$\delta, \sigma_j = 10$	32 to 59	8 to 31	-12 to 7	-32 to -13	-60 to -33
$\delta, \sigma_j = 11$	32 to 65	8 to 31	-12 to 7	-34 to -13	-66 to -35
$\delta, \sigma_j = 12$	36 to 71	12 to 35	-12 to 11	-36 to -13	-72 to -37
$\delta, \sigma_j = 13$	40 to 75	16 to 39	-16 to 15	-40 to -17	-76 to -41
$\delta, \sigma_j = 14$	40 to 81	16 to 39	-16 to 15	-44 to -17	-82 to -45
$\delta, \sigma_j = 15$	48 to 85	16 to 47	-16 to 15	-48 to -17	-88 to -49



(a) Radix-2 Basic RDS (b) Radix-4 Basic RDS (c) Radix-4 Optimized RDS

Figure 2: Selection regions for the Root Digit Selector (RDS). The y-axis is the truncated residual estimate τ_j and the x-axis is the truncated partial root estimate σ_j. The plots show how the choice of root digit s_{j+1} depends on these estimates.

5 Conclusion

Digit recurrence algorithms provide an efficient framework for implementing division and square root in hardware. They address a central tension in iterative arithmetic: exact comparisons and full-precision state updates can limit speed, but overly aggressive approximation can break correctness. The core idea that resolves this tension is *redundancy*. By selecting digits from a redundant set (with $\rho > \frac{1}{2}$), SRT-style methods create overlapping selection regions. This overlap permits the next digit to be chosen using fast, truncated estimates of the residual (and, for square root, the partial root), avoiding a full-width carry-propagating addition in the critical path.

This tutorial derived the fundamental recurrence relations for division and square root, established the containment conditions that guarantee convergence, and showed how those conditions drive practical digit-selection design. The continuity condition provides the key bridge from theory to implementation: it determines how much operand truncation can be tolerated while still guaranteeing that overlap covers uncertainty. The Radix-2 and Radix-4 case studies illustrated a complete design workflow, from interval derivation through parameter selection to final selection logic.

Square root follows the same broad methodology as division but introduces additional complexity because its bounds depend on the evolving partial root S_j and include iteration-dependent terms. As the Radix-4 example showed, these terms can shrink overlap in early iterations, requiring either greater precision or special startup logic. This highlights the trade-offs among radix (performance), selection complexity, and implementation cost.

Ultimately, digit recurrence methods exemplify a practical design paradigm in computer arithmetic: exploit algorithmic structure (redundancy and overlap) to simplify and accelerate hardware. They are not merely of theoretical interest but remain a cornerstone of high-performance arithmetic units in modern processors.